



boolean

Strings



Strings

Let's move on to the next data type: **strings**!

Strings are simply how we **represent text** within our programs, from a **single character** to a **whole book**. In JS, we can create a string by wrapping our characters in **single**, **double** or **back quotes**

```
"hey there!" // ✓
```

```
'hey there!' // ✓
```

```
`hey there!` // ✓
```

Q: Why might you prefer one over the other?



Strings

Double and **single quote** strings are pretty much the same thing, but beware of:

```
'Hi I'm Nicolas'
```

Uncaught SyntaxError: Unexpected identifier

```
"She said "yes"!"
```

Uncaught SyntaxError: Unexpected identifier

See what happened there? One way to fix this is to use another kind of quotes to wrap it:

```
"Hi, I'm Nicolas"
```

```
'She said "yes"!'
```

Another way is to **escape** the quotes:

```
'Hi, I\'m Nicolas'
```

Using the **backslash**, we can let JS know that the character that comes next should not be taken literally, but it's **just another character**.



Strings

We can add strings together by using the **plus “+”** operator. This is called **concatenation**:

```
"Hello" + "there!"
```

Q: What will be the result of that expression?

Here's two ways you could fix it:

```
"hello" + " " + "there!"
```

```
"hello " + "there!"
```

But why do we care about the plus when we could just do the whole thing in **a single string**?

Well, take a look at this:

```
const teacher = "Nicolas"
```

```
"Hello" + teacher + "!"
```

Adding variables to a string is called **interpolation**, and you'll be doing this **a lot**!



Strings

One **limitation** of these kind of strings is they won't allow you to go to the next line, meaning this will happen:

```
"nice
one"
=> Uncaught SyntaxError: Invalid or unexpected token
```

There's **two ways** to fix this: we can **escape** the **newline character**:

```
"nice \
one"
=> "nice one"
```

Or **concatenate** separate strings:

```
"nice " +
"one"
=> "nice one"
```



Strings

If you actually wanted to go to the next line within your string, and not just the code, you can use the special `\n` newline character:

```
"nice\none"
```

```
=> "nice  
    one"
```

Top tip

There are several special characters that need to be escaped in strings, go check them out on [MDN](#)

MDN is an awesome documentation site run by **Mozilla**, the people behind **Firefox**



Strings

As a side note, while **some languages** are strict and **won't let you mix types together**, **JavaScript** is **totally cool with it**:

```
const age = 30
const name = "Maria"
name + " is " + age + " years old!"
=> "Maria is 30 years old!"
```

Although this may lead to unexpected behaviour in some cases, so **use with care** and **always test**!

```
"Hey " + {} + "!"
=> "Hey [object Object]!"
```



Strings

Finally, we have strings wrapped in **back quotes** (or **backticks**), the technical term for this kind of string is **template literal**:

```
`Hello there, person!`
```

These are a **modern** addition to JavaScript (2015), and they behave like regular strings for the most part, but they have a couple of cool super powers! Watch this:

```
`nice  
one`  
=> "nice  
  one"
```

```
const yourName = "Nicolas"  
`Hey there, ${yourName}!`  
=> "Hey there, Nicolas!"
```

In short, they accept **newlines** without having to use `\n`, and they also have a **special interpolation syntax `${}`**, which can make things look much cleaner sometimes!



Strings

Strings have a TON of useful **properties**, here's a couple of really useful ones, but feel free to explore more in the console!

```
"Nicolas".length
```

```
=> 7
```

```
"Nicolas".toUpperCase()
```

```
=> "NICOLAS"
```

Q: What do they do? What types do they return? Is the syntax for getting both properties the same?

Top tip

If you are ever unsure about what something does in JavaScript, google the **mdn + term**

i.e.: [mdn touppercase](#) (properties are case-sensitive, but google isn't!)



Strings

To wrap up strings for now, know that you can also **compare** strings, like you do with numbers:

```
"Nicolas" === "Nicolas"
```

```
=> true
```

```
"Nicolas" === "nicolas"
```

```
=> false // it's case-sensitive
```

```
"a" > "b"
```

```
=> false
```

```
"b" > "a"
```

```
=> true
```

```
"a" > "A"
```

```
=> true // mmm, why? ASCII yourself that question
```